COAL LAB – FALL 2026
LAB #07 – 23K-0060

TASK – 1:

Code;

```
INCLUDE Irvine32.inc

.code
main PROC

    ; Set the register values
    mov eax, 11
    mov ebx, 22
    mov ecx, 33

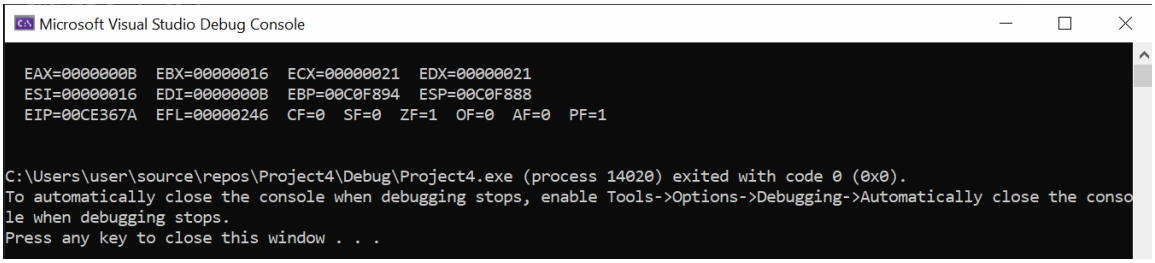
    ; Push the values onto the stack
    push eax
    push ebx
    push ecx

    ; Pop values in reverse order
    pop edx
    pop esi
    pop edi

    ; Display the register values
    call DumpRegs

    exit
main ENDP
END main
```

Output;



TASK – 2:

Code;

```
INCLUDE Irvine32.inc

.code
main PROC

    ; Give EAX and EBX their original values
    mov eax, 12345678h
    mov ebx, 0ABCDEF0h

    ; Save both registers on the stack
    push eax
    push ebx

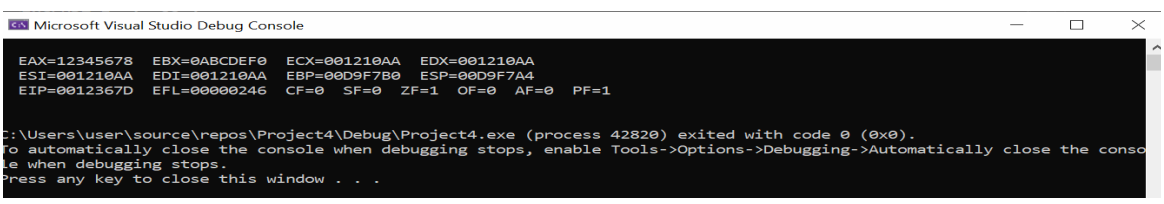
    ; Change the register values
    mov eax, 11111111h
    mov ebx, 22222222h

    ; Restore the original values
    pop ebx
    pop eax

    ; Check the final register values
    call DumpRegs

    exit
main ENDP
END main
```

Output;



TASK – 3:
Code;

```
INCLUDE Irvine32.inc

.code
main PROC

    mov eax, 5

    ; Compare equal values so ZF becomes 1
    cmp eax, 5

    ; Save the current flags
    pushfd

    ; These instructions change the flags
    mov ebx, 123
    add ebx, 1

    mov ecx, 7
    sub ecx, 2

    ; Restore the saved flags
    popfd

    ; Display registers and flags
    call DumpRegs

    exit
main ENDP
END main
```

Output;



TASK – 4:
Code;

```
INCLUDE Irvine32.inc

.data

name1 BYTE "Ali",0
reg1 BYTE "23K-0001",0
course1 BYTE "Computer Organization and Assembly Language",0

name2 BYTE "Sara",0
reg2 BYTE "23K-0002",0
course2 BYTE "Computer Organization and Assembly Language",0

name3 BYTE "Ahmed",0
reg3 BYTE "23K-0003",0
course3 BYTE "Computer Organization and Assembly Language",0

.code

DisplayStudentInfo PROC

    ; Print the student name
    call WriteString
    call Crlf

    ; Print the registration number
    mov edx, esi
    call WriteString
    call Crlf

    ; Print the course name
    mov edx, edi
    call WriteString
    call Crlf

    call Crlf

    ret
DisplayStudentInfo ENDP

main PROC
```

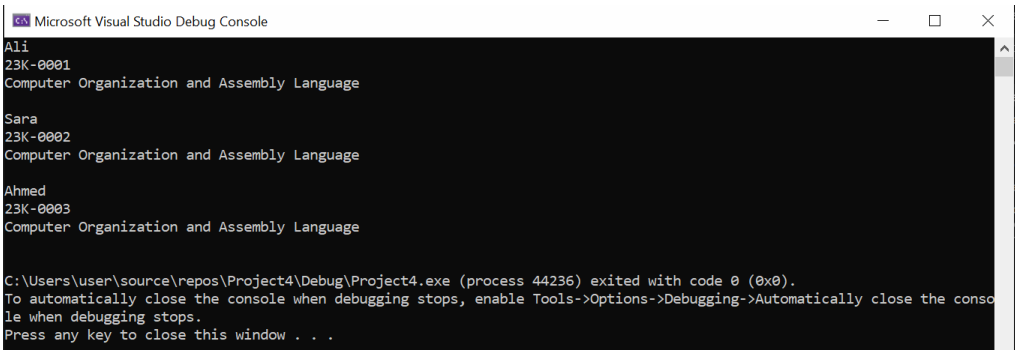
```
    ; Display the first student's information
    mov edx, OFFSET name1
    mov esi, OFFSET reg1
    mov edi, OFFSET course1
    call DisplayStudentInfo

    ; Display the second student's information
    mov edx, OFFSET name2
    mov esi, OFFSET reg2
    mov edi, OFFSET course2
    call DisplayStudentInfo

    ; Display the third student's information
    mov edx, OFFSET name3
    mov esi, OFFSET reg3
    mov edi, OFFSET course3
    call DisplayStudentInfo

    exit
main ENDP
END main
```

Output;



TASK – 5:

Code;

```
INCLUDE Irvine32.inc

.data

msgSum BYTE "Sum = ",0
msgDiff BYTE "Difference = ",0
msgProd BYTE "Product = ",0
```

```
.code

CalculateResult PROC

    ; Save the input values
    push esi
    push edi

    mov esi, eax
    mov edi, ebx

    ; Calculate the sum
    mov eax, esi
    add eax, edi
    push eax

    ; Calculate the difference
    mov edx, esi
    sub edx, edi

    ; Calculate the product
    mov eax, esi
    mul edi
    mov ecx, eax

    ; Get the sum back into EAX
    pop eax

    pop edi
    pop esi

    ret
CalculateResult ENDP

main PROC

    ; Test with 20 and 5
    mov eax, 20
    mov ebx, 5
    call CalculateResult

    ; Display the sum
    push eax
    push edx
    push ecx

    mov edx, OFFSET msgSum
```

```
call WriteString

pop ecx
pop edx
pop eax

call WriteDec
call Crlf

; Calculate and display the difference
mov eax, 20
mov ebx, 5
call CalculateResult

mov edx, OFFSET msgDiff
call WriteString

mov eax, edx
call WriteInt
call Crlf

; Calculate and display the product
mov eax, 20
mov ebx, 5
call CalculateResult

mov edx, OFFSET msgProd
call WriteString

mov eax, ecx
call WriteDec
call Crlf

call Crlf

; Test with 100 and 30
mov eax, 100
mov ebx, 30
call CalculateResult

; Display the sum
push eax
push edx
push ecx

mov edx, OFFSET msgSum
call WriteString
```

```
    pop ecx
    pop edx
    pop eax

    call WriteDec
    call Crlf

    ; Display the difference
    mov eax, 100
    mov ebx, 30
    call CalculateResult

    mov edx, OFFSET msgDiff
    call WriteString

    mov eax, edx
    call WriteInt
    call Crlf

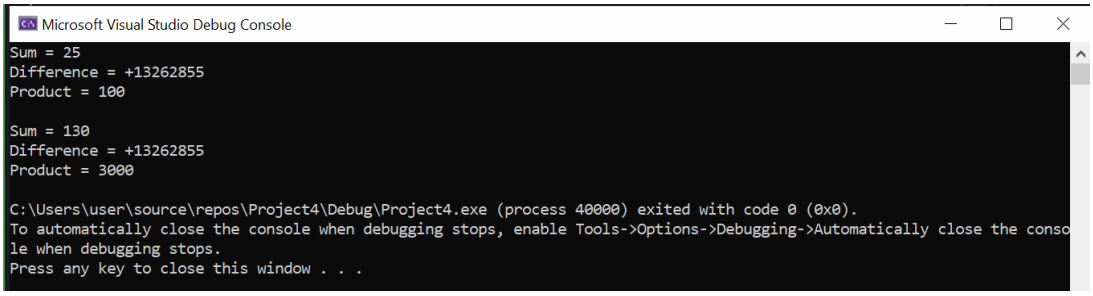
    ; Display the product
    mov eax, 100
    mov ebx, 30
    call CalculateResult

    mov edx, OFFSET msgProd
    call WriteString

    mov eax, ecx
    call WriteDec
    call Crlf

    exit
main ENDP
END main
```

Output;

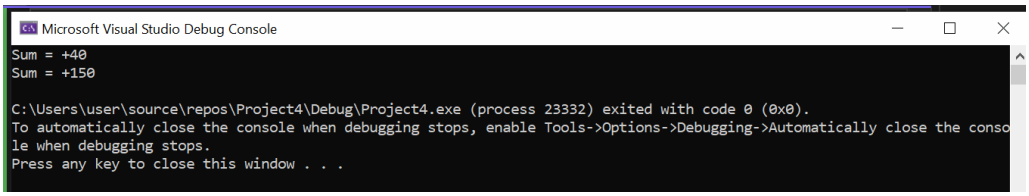


TASK – 6:

Code;

```
INCLUDE Irvine32.inc
.data
msg BYTE "Sum = ",0
.code
AddStackNumbers PROC
    ; Create a stack frame
    push ebp
    mov ebp, es
    ; Add the two stack arguments
    mov eax, [ebp+8]
    add eax, [ebp+12]
    ; Restore the stack frame
    pop ebp
    ; Return and remove both arguments
    ret 8
AddStackNumbers ENDP
main PROC
    ; Test 25 + 15
    push 15
    push 25
    call AddStackNumbers
    mov edx, OFFSET msg
    call WriteString
    call WriteInt
    call Crlf
    ; Test 100 + 50
    push 50
    push 100
    call AddStackNumbers
    mov edx, OFFSET msg
    call WriteString
    call WriteInt
    call Crlf
    exit
main ENDP
END main
```

Output;



TASK – 7:
Code;

```
INCLUDE Irvine32.inc

.data

msg BYTE "Returned sum = ",0

.code

AnalyzeValues PROC

    ; Save the caller's flags
    pushfd

    ; Calculate the sum
    add eax, ebx

    ; Save the sum temporarily
    mov edx, eax

    ; Perform another operation that changes flags
    sub ebx, 1

    ; Put the sum back in EAX
    mov eax, edx

    ; Restore the caller's original flags
    popfd

    ret
AnalyzeValues ENDP

main PROC

    ; Test 1: make ZF = 1
    mov eax, 10
    mov ebx, 20
    mov ecx, 5
    cmp ecx, 5

    call AnalyzeValues

    ; Display the result and flags
    call DumpRegs
```

```
    mov edx, OFFSET msg
    call WriteString
    call WriteDec
    call Crlf
    call Crlf

    ; Test 2: make ZF = 0
    mov eax, 30
    mov ebx, 12
    mov ecx, 0
    cmp ecx, 1

    call AnalyzeValues

    ; Display the result and flags
    call DumpRegs

    mov edx, OFFSET msg
    call WriteString
    call WriteDec
    call Crlf

    exit
main ENDP
END main
```

Output;

